Operating System Structures

BIG QUESTION

Your program can't touch the disk. So how does it save a file?

What the OS is made of, what it offers programs, how programs ask for it, and how the whole thing is organised.

Chapter 3 of the notebook

Today's route

What we'll cover today

1. **System components**
   The OS's departments
2. **OS services**
   What programs can expect
3. **System calls**
   How programs ask
4. **System structure**
   Simple vs layered design

Part 01

System components

THINK FIRST

If you were building an OS from scratch, which jobs would you give to separate departments?

discuss for one minute before the next slide

The big picture

Eight departments of the OS

- Process management
- Main-memory management
- File management
- I/O system management
- Secondary-storage management
- Networking
- Protection
- Command interpreter

Component 1

Process management

↺ Recall · defined in Ch 4

A process is a program in execution. It needs CPU time, memory, files and I/O devices to do its job.

1. **Create & delete** processes
2. **Suspend & resume** processes
3. **Communicate** between processes

Component 2

Main-memory management

↺ Recall · hardware in Ch 2

Memory is a large array of bytes, each with its own address. It is volatile: it loses its contents on failure.

1

Track usage

Which parts are in use, and by whom

2

Decide who loads

Which processes to load when space frees up

3

Allocate & free

Give and take back memory as needed

Component 3

File management



What is a file?

A collection of related information defined by its creator. Usually programs and data.

1. Create and delete files
2. Create and delete directories
3. Provide basic operations on files and folders
4. Map files onto secondary storage
5. Back up files on stable (non-volatile) storage

Component 4

I/O system management: hide the device details



Device drivers

- One driver per kind of device
- It translates the OS's requests into commands that device understands
- Like an interpreter between the OS and the device



Spooling

Buffer output in memory or on disk so a slow device can take it at its own pace.



Real life

The office printer queue: everyone "prints" instantly, and the printer works through the pile one job at a time.

Component 5

Secondary-storage management

↺ Recall · hardware in Ch 2

Main memory is volatile and too small, so disks back it up. Disks are the main on-line storage for programs and data.

1. **Free-space management**
2. **Storage allocation**
3. **Disk scheduling**

Components 6 and 7

Networking and protection

↺ Recall · distributed systems in Ch 1

Processors without shared memory or clock, connected by a network.



Networking component

The OS talks over the network using a **protocol**, and gives users access to remote resources.



Protection

- Controls access by programs or users to system resources
- Must tell authorised from unauthorised use
- Must specify the controls to impose

Component 8

The command interpreter, a.k.a. the shell



Its only job

Read your next command, carry it out, then wait for the next one.



Also called

The **command-line interpreter**, or the **shell** in UNIX.



Every command lands somewhere

- Run a program → process management
- Open a file → file management
- Print → I/O management

Pop quiz • click to reveal

Memory just freed up. Which component decides which process gets loaded next?

A File management

B Main-memory management

C Command interpreter

D Networking

Pop quiz • click to reveal

Memory just freed up. Which component decides which process gets loaded next?

A File management

B Main-memory management

C Command interpreter

D Networking

Answer B. Deciding which processes to load when memory becomes available is a main-memory management job.

Part 02

Operating system services

THINK FIRST

Imagine you are a program. What would you expect the OS to do for you?

discuss for one minute before the next slide

What programs get

Five services



Program execution

Load a program into memory and run it.



I/O operations

Programs can't do I/O directly, so the OS does it for them.



File-system manipulation

Read, write, create and delete files.



Communications

Processes exchange information, locally or over a network.



Error detection

Catch errors in the CPU, memory, I/O devices or user programs.

How communication works → Ch 4 (IPC)

How errors are trapped → Ch 2

Components are the inside view. Services are the outside view.

Same OS, two angles: how it is organised vs what it offers to programs.

Part 03

System calls

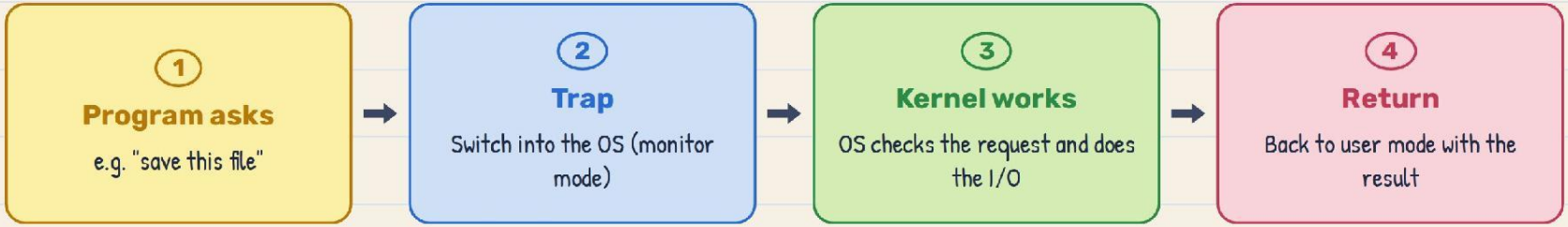
THINK FIRST

A program runs in user mode and can't touch hardware. How does it save a file?

discuss for one minute before the next slide

The interface

A system call is how a program asks the OS



↺ Recall • traps in Ch 2

A request from a user program to the OS is a software interrupt: a trap.

Example

Copying one file takes many requests to the OS

1. Open the original file
2. Create the new file
3. Read a piece of the original
4. Write that piece to the new file
5. Repeat until everything is copied
6. Close both files



Each step is a system call

The program can't touch the disk itself, so it asks the OS every time.

one simple "copy" = dozens of system calls

Five families

Types of system calls



Process control

Start, stop, or wait for a program.



File management

Create, open, read, write, close, delete files.



Device management

Request and release a device (printer, disk…).



Information

Get or set the time, date, system data.



Communications

Send and receive messages between processes.

how processes talk → Chapter 4

Pop quiz • click to reveal

"Read a piece of the original file" belongs to which family of system calls?

A Process control

B File management

C Information maintenance

D Communications

Pop quiz • click to reveal

"Read a piece of the original file" belongs to which family of system calls?

A Process control

B File management

C Information maintenance

D Communications

Answer B. Reading from a file is a file-management system call.

Part 04

System structure

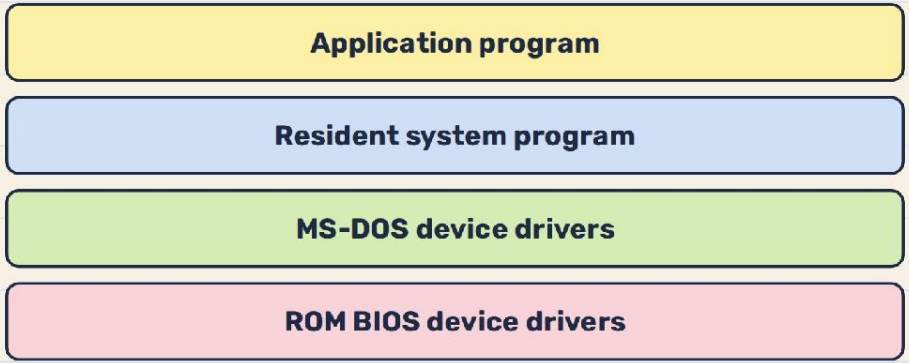
THINK FIRST

Would you build a house as one giant room, or floor by floor?

discuss for one minute before the next slide

Simple structure

MS-DOS: small, fast, unprotected



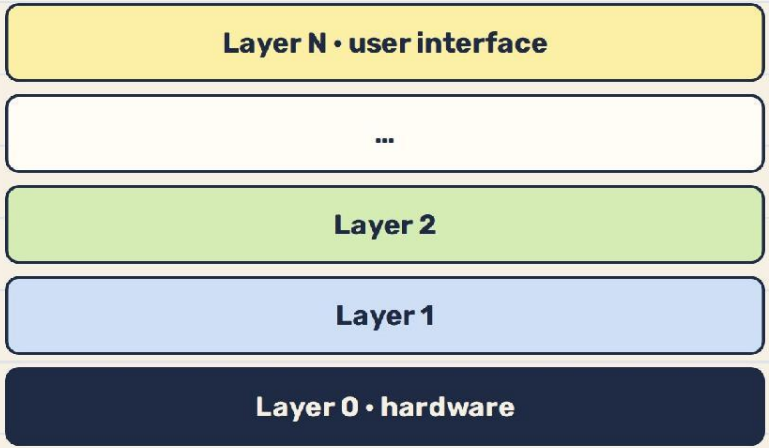
The catch

Interfaces and levels are not well separated. An application can call the device drivers, or even the hardware, directly.

one buggy program → the whole system crashes

Layered approach

Each layer only uses the layers below



The rule

The OS is split into levels. Layer 0 is the hardware, layer N is the user interface. Each layer uses only the functions and services of lower layers.

Real life

A request climbs a building floor by floor; nobody skips a floor.

Layered approach

Clean, but slower



Advantages

- **Modularity:** each layer is a unit
- **Debugging:** test layer by layer, bottom up
- **Modification:** change one layer without touching the others



Disadvantage

- **Layering overhead:** a system call may pass through several layers, and each adds time

Pop quiz • click to reveal

What is the main drawback of the layered approach?

A It can't be debugged

B Each layer adds overhead to a system call

C Applications can reach hardware directly

D It has no user interface

Pop quiz • click to reveal

What is the main drawback of the layered approach?

A. It can't be debugged

B. Each layer adds overhead to a system call

C. Applications can reach hardware directly

D. It has no user interface

Answer B. Passing through several layers costs time on every system call. Direct hardware access is the MS-DOS problem, not the layered one.

Wrap-up

What to remember

OS JOURNAL · CHAPTER 3

- ✓ Eight components: process, memory, files, I/O, storage, networking, protection, shell.
- ✓ A system call is a trap from a program into the OS.
- ✓ MS-DOS: simple, but apps can bypass layers.
- ✓ Services are what programs get: execution, I/O, files, communication, error detection.
- ✓ Five families: process, file, device, information, communications.
- ✓ Layered: modular and debuggable, at the cost of overhead.

Next up *Chapter 4 · Processes: what a running program really is*